```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

```
wine_quality = pd.read_csv('/content/winequality-red.csv')
```

```
wine_quality.head()
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	p
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.5
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.2
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.2
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.1
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.5

Next steps: Generate code with `wine_quality` New interactive sheet

```
wine_quality.isnull().sum()
```

	0
fixed acidity	0
volatile acidity	0
citric acid	0
residual sugar	0
chlorides	0
free sulfur dioxide	0
total sulfur dioxide	0
density	0
pH	0
sulphates	0
alcohol	0
quality	0

dtype: int64

```
wine_quality.shape
```

```
(1599, 12)
```

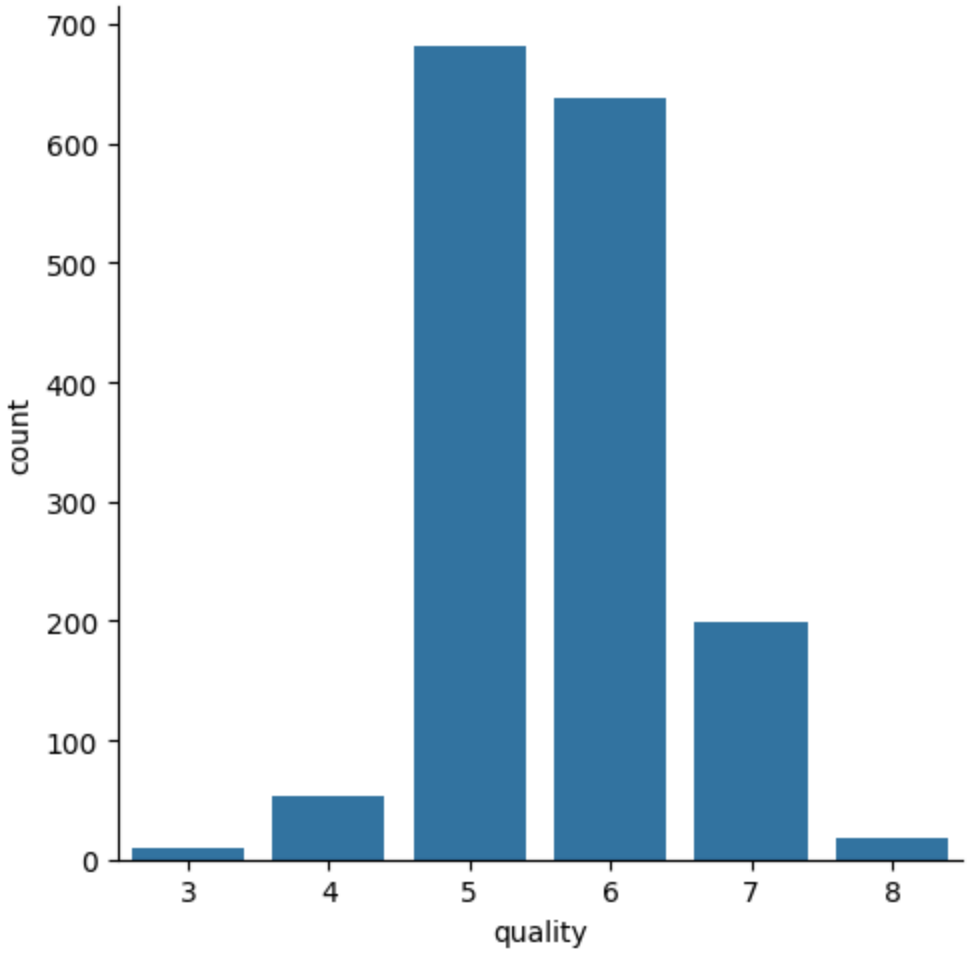
```
wine_quality.describe()
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	fre sulfu dioxid
count	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000	1599.00000
mean	8.319637	0.527821	0.270976	2.538806	0.087467	15.87492
std	1.741096	0.179060	0.194801	1.409928	0.047065	10.46015
min	4.600000	0.120000	0.000000	0.900000	0.012000	1.00000
25%	7.100000	0.390000	0.090000	1.900000	0.070000	7.00000
50%	7.900000	0.520000	0.260000	2.200000	0.079000	14.00000
75%	9.200000	0.640000	0.420000	2.600000	0.090000	21.00000
max	15.900000	1.580000	1.000000	15.500000	0.611000	72.00000

```
# number of values for each quality

sns.catplot(x='quality', data=wine_quality, kind='count')
```

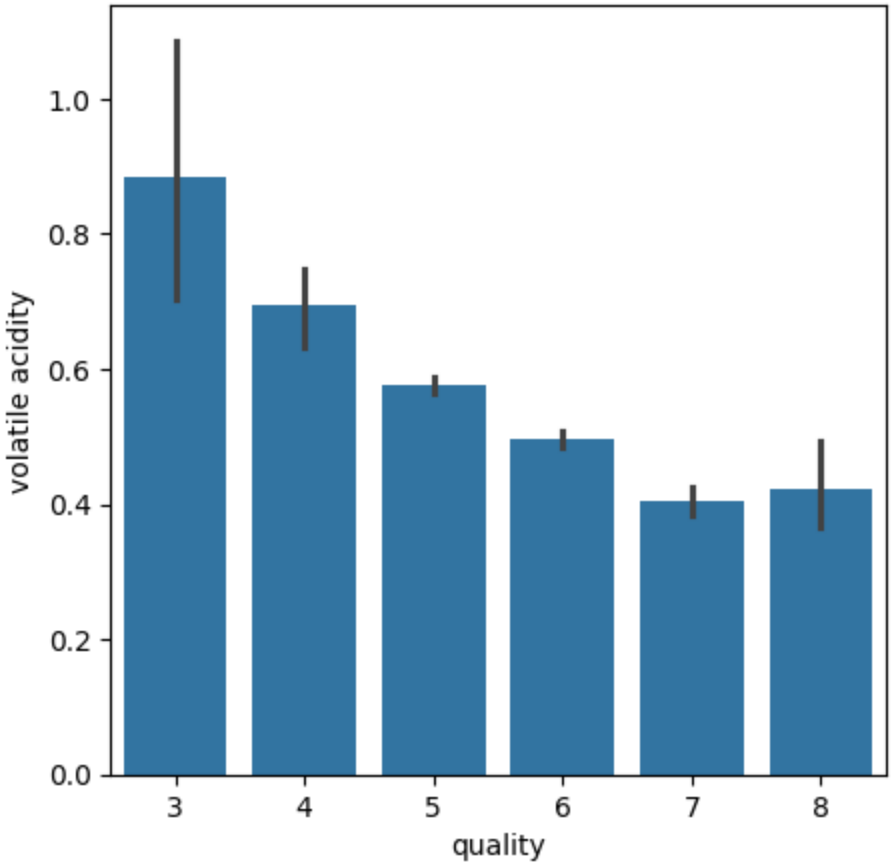
```
<seaborn.axisgrid.FacetGrid at 0x78d52fb29190>
```




```
# volatile acidity vs Quality

plot = plt.figure(figsize=(5,5))
sns.barplot(x='quality', y='volatile acidity', data=wine_quality)
```

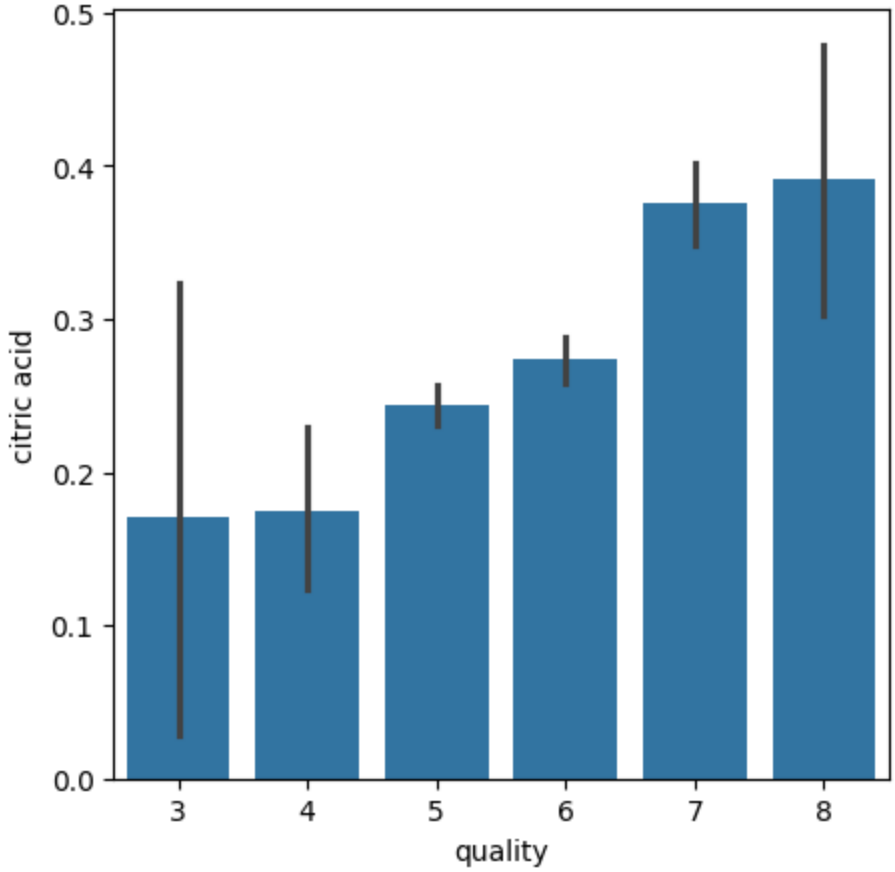
```
<Axes: xlabel='quality', ylabel='volatile acidity'>
```




```
# citric acid vs Quality

plot = plt.figure(figsize=(5,5))
sns.barplot(x='quality', y='citric acid', data=wine_quality)
```

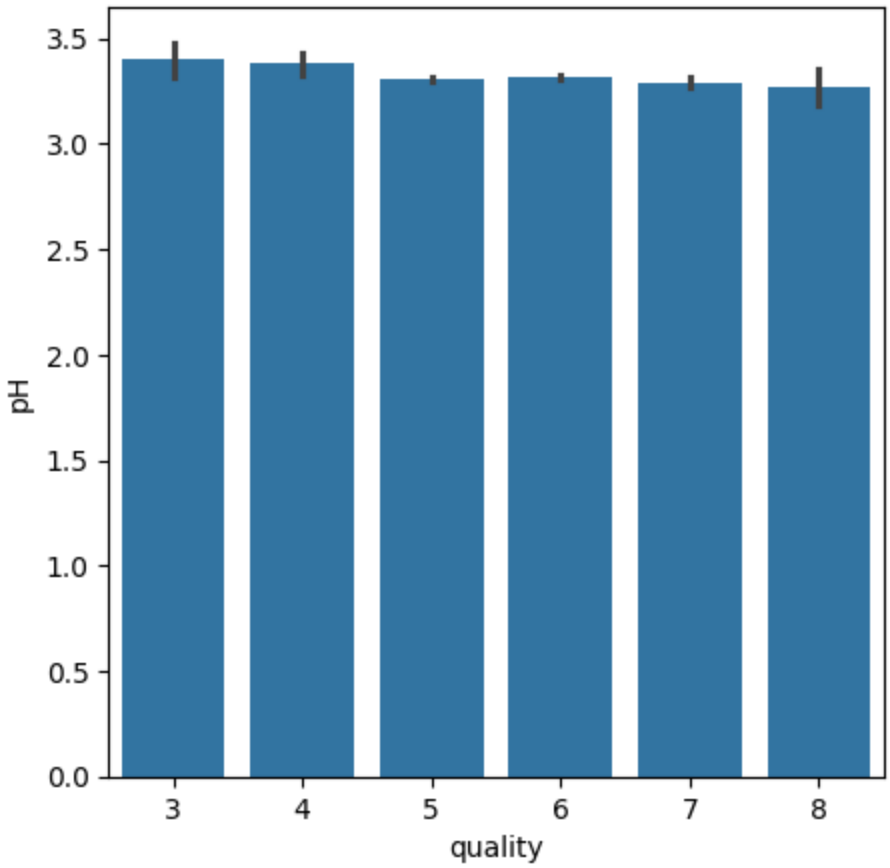
```
<Axes: xlabel='quality', ylabel='citric acid'>
```




```
# pH vs Quality

plot = plt.figure(figsize=(5,5))
sns.barplot(x='quality', y='pH', data=wine_quality)
```

```
<Axes: xlabel='quality', ylabel='pH'>
```



Correlation

```
correlation = wine_quality.corr()
```

```
print(correlation)
```

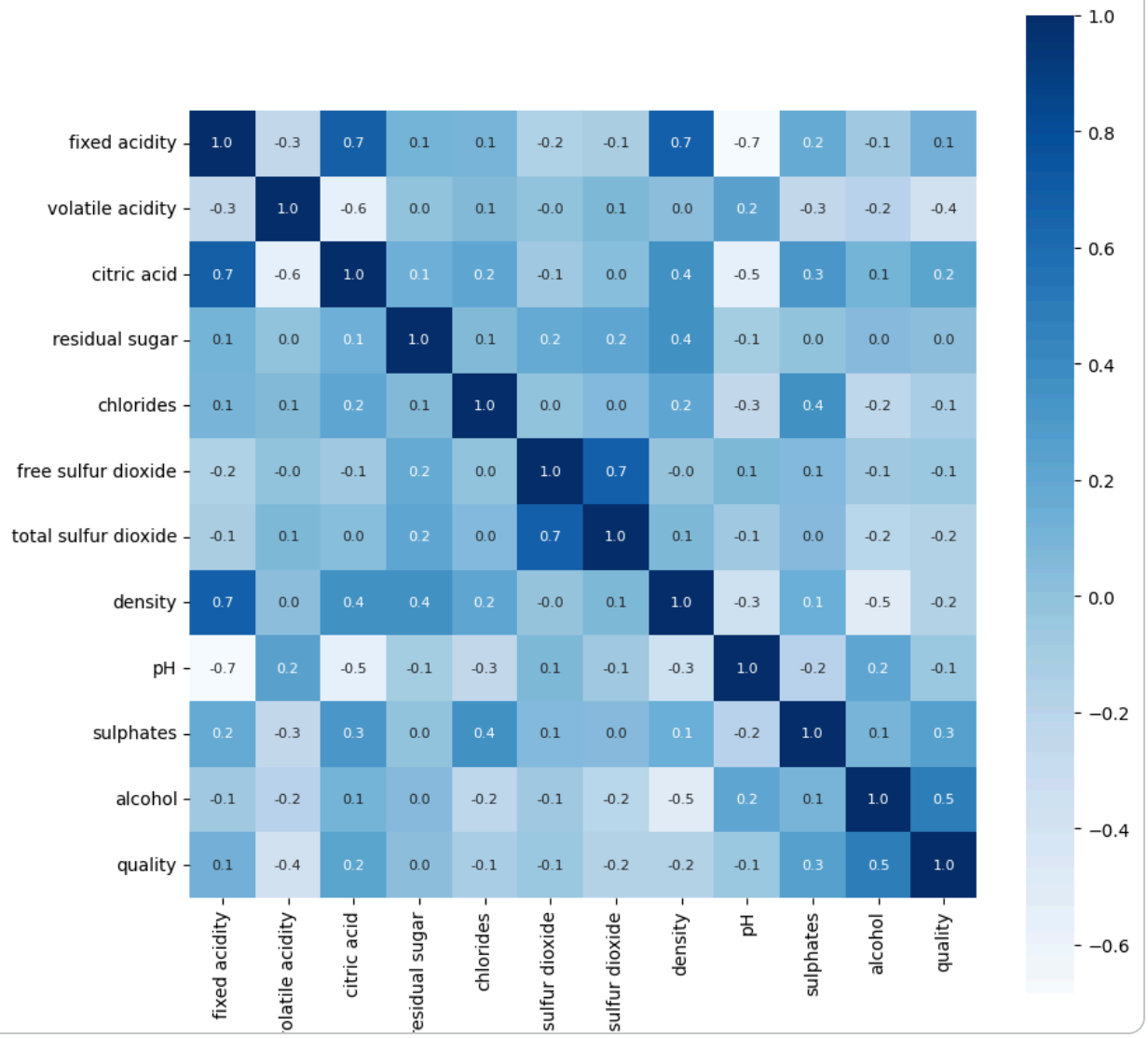
```
                      fixed acidity  volatile acidity  ...   alcohol   quality
fixed acidity              1.000000         -0.256131  ... -0.061668  0.124052
volatile acidity          -0.256131          1.000000  ... -0.202288 -0.390558
citric acid                0.671703         -0.552496  ...  0.109903  0.226373
residual sugar             0.114777          0.001918  ...  0.042075  0.013732
chlorides                  0.093705          0.061298  ... -0.221141 -0.128907
free sulfur dioxide       -0.153794         -0.010504  ... -0.069408 -0.050656
total sulfur dioxide      -0.113181          0.076470  ... -0.205654 -0.185100
density                    0.668047          0.022026  ... -0.496180 -0.174919
pH                        -0.682978          0.234937  ...  0.205633 -0.057731
sulphates                  0.183006         -0.260987  ...  0.093595  0.251397
alcohol                   -0.061668         -0.202288  ...  1.000000  0.476166
quality                    0.124052         -0.390558  ...  0.476166  1.000000

[12 rows x 12 columns]
```

```
# heatmap to analyze the correlation
```

```
plt.figure(figsize=(10,10))
sns.heatmap(correlation, cbar=True, square=True, fmt='.1f', annot=True, annot_k
```

```
<Axes: >
```



Data Preprocessing

```
X = wine_quality.drop('quality', axis=1)

# label binarization
# 1 = Good Quality
# 0 = Bad Quality
Y = wine_quality['quality'].apply(lambda y_value: 1 if y_value>=7 else 0)
```

```
print(Y)
```

```
0       0
1       0
2       0
3       0
4       0
```

```
        ..
1594    0
1595    0
1596    0
1597    0
1598    0
Name: quality, Length: 1599, dtype: int64
```

Train & Test Split

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random
```

```
print(Y.shape, Y_train.shape, Y_test.shape)
```

```
(1599,) (1279,) (320,)
```

Model Training

```
# Random Forest Clasifier
```